



University of Stuttgart
Institute for Signal Processing and System Theory
Professor Dr.-Ing. B. Yang



Bachelorarbeit S1509

Implementation of an Electromyography Processing Pipeline from Torch model to TinyML on resource-constrained microcontroller platforms

Arbeitstitel, to be defined (TBD)

Author: Andrew Fady Fahmy

Date of work begin: 01/03/2026

Date of submission: 30/06/2026

Supervisor: Jonas Koellner, Prof. Dr. Turker
Ince

Keywords: sEMG, TinyML, Digital Signal
Processing, Deep Learning, re-
gression, Limited-Resource Mi-
crocontroller Platforms

Abstract TBD

Contents

1. Basics and Foundations	1
1.1. EMG and Deep Learning Pipelines for Bioelectric Signal Processing	1
1.1.1. Bioelectric Signals and Human Machine Interaction (HMI)	1
1.1.2. Electromyography (EMG) Basics	1
1.1.3. EMG Signal Preprocessing and Feature Extraction	2
1.1.4. Deep Learning (DL) Pipelines for Bioelectric Signal Processing	2
1.1.5. Deep Learning (DL) for EMG Signal Processing	2
1.2. TinyML and Resource-Constrained Microcontroller Platforms	2
1.2.1. Microcontrollers	2
1.2.2. Limitations and Constraints of Microcontroller Platforms	2
1.2.3. TinyML Overview	2
1.2.4. TinyML Frameworks and tools	4
1.2.5. Applications of TinyML in Bioelectric Signal Processing	4
2. State of The Art (SoTA)	5
2.0.1. EMG Signal Processing Pipelines on Microcontroller Platforms	5
2.0.2. EMG Signal Processing Pipelines with TinyML Frameworks	5
3. Materials and Methods	7
3.1. Dataset Description and Preprocessing	7
3.1.1. NinaPro DB2 Dataset	7
3.1.2. Preprocessing and Feature Extraction	7
3.2. Deep Learning Model Architecture	7
3.3. Hardware	7
3.3.1. ESP32-S3 Specifications	7
3.4. Quantization	7
3.4.1. Quantization Types and Schemes	8
3.4.2. Quantization Scheme Used	10
3.5. Software and Tools	10
3.5.1. ESP-NN Library	10
3.5.2. LiteRT (Tensorflow Lite) Framework	11
3.5.3. LabStreaming Layer (LSL) for Data Acquisition and Real-Time Processing	11
4. Results	13
4.1. Comparison of ML Ops Performance on the ESP32-S3 using Random Weights under different Quantization Schemes	13
4.2. Comparison of different Model Configurations	13

5. Discussion	17
5.1. Software Challenges	17
5.2. LSTM Compatibility and Overhead	17
6. Conclusion	19
6.1. Conclusion	19
6.2. Limitations	19
6.3. Future Work	19
A. Additionally	21
List of Figures	23
List of Tables	25
Bibliography	27

1. Basics and Foundations

1.1. EMG and Deep Learning Pipelines for Bioelectric Signal Processing

1.1.1. Bioelectric Signals and Human Machine Interaction (HMI)

Bioelectric signals are simply defined as a biological signal that originates from a human body, they are the sum of action potentials of cells at an anatomical site such as the heart, brain, or skeletal muscle [?]. These signals have been used extensively in Human Machine Interaction as they provide key insights about human's physiological activity as well as presenting a more natural interaction medium between humans and computers. Although our primary focus in this work is about Electromyography (EMG) signals, we will frequently discuss other commonly researched bioelectric signals such as EEG (Electroencephalogram) and ECG (Electrocardiogram) signals given the abundance of useful HMI research conducted using these 2 domains.

1.1.2. Electromyography (EMG) Basics

Electromyography is the recording of electrical activity in muscle. These electrical impulses originate from the spinal cord and propagate all the way to the peripheral nerves activating muscular contraction [?]. The motor neurons and the muscle fibers they innervate are defined as Motor Units which are recruited by these electrical impulses producing electrical signals. Professional electromyographers analyze characteristics features of these signals (i.e waveforms, firing rates, frequency of abrupt discharges, etc) deriving useful diagnostic information to identify abnormalities and different neural and muscle diseases

The recording of EMG signals is mainly classified into two main categories. The first is intramuscular electromyography (iEMG) which is an invasive technique that includes inserting concentric needle electrodes inserted directly into muscle. The other is surface electromyography (sEMG) which is a non-invasive technique involving the placement of adhesive surface electrodes on skin surface. While iEMG has proven to be more effective in diagnosing medical conditions and abnormalities, sEMG dominate the HMI and gesture recognition sector due to its non-invasive nature, wider accessibility without the need of trained experts and user comfort [?].

In this study, we exclusively focus on sEMG data for the reasons mentioned above

1.1.3. EMG Signal Preprocessing and Feature Extraction

Raw EMG signals suffer from many issues that obscure its insightful value.

1.1.4. Deep Learning (DL) Pipelines for Bioelectric Signal Processing

1.1.5. Deep Learning (DL) for EMG Signal Processing

1.2. TinyML and Resource-Constrained Microcontroller Platforms

1.2.1. Microcontrollers

A Microcontroller unit (MCU) is a self-contained computer implemented on a single integrated circuit. It includes necessary units for regular computation like a fully functioning CPU, integrated memory (RAM), Permanent FLASH memory, IO pins, etc. The adoption of MCUs have seen rapid growth in recent years due to the development of IoT systems (Internet of Things) that needs multiple MCU devices to control, process and compute useful insights from on-ground sensors and communicate it effectively through dedicated channels and networks for further processing and decision-making.

Many key vendors include Arduino (well known for their UNO and Nano modules), Espressif Systems who produce ESP32 MCUs, STMicroelectronics with STM32s, etc. The MCU landscape is constantly shifting every day with new innovations.

1.2.2. Limitations and Constraints of Microcontroller Platforms

Given their cheap cost, small footprint and low-power consumption, MCUs come with many constraints and downgrades relative to personal and heavy computing devices.

For starters, on-chip memory is 3 orders of magnitude smaller than mobile devices and 5-6 orders of magnitude smaller than cloud GPUs [?]. In addition, their CPU clock rate are in MHz range compared to modern personal-use processors which are in the GHz range. Another important aspect is the lack of GPUs or Parallel Processing Units which severely limits the MCU at processing SIMD instructions and vector functions which traditionally limited their use in Graphics and ML fields. Furthermore, Software development for MCUs is notoriously convoluted due to fragmented vendor-specific sdks and frameworks, the wide use of deprecated software in order to accommodate MCU needs and lack of continuous support and maintenance for development frameworks.

1.2.3. TinyML Overview

In the past, the only solution for IoT ML and DL processing was CloudML. This process involved using the communication capabilities of MCUs, e.g WIFI, in order to send raw

Table 1.1.: Comparison of Commercially Available MCUs for Edge ML Applications

MCU	Processor (CPU / cores / clock)	WiFi/BT	Memory & FPU	NN / SIMD	Cost (USD)
ESP32	Xtensa 32-bit LX6, 2 cores, 240 MHz	Yes	SRAM 520 KB PSRAM 8 MB Flash 16 MB ext. FPU SP	ESP-DL / ESP-NN	\$3–8
ESP32-S3	Xtensa 32-bit LX7, 2 cores, 240 MHz	Yes	SRAM 520 KB PSRAM 8 MB Flash 16 MB ext. FPU SP	ESP-DL / ESP-NN	\$4–10
Pi Pico (RP2040)	Arm Cortex-M0+, 2 cores, 133 MHz	W only	SRAM 264 KB PSRAM ext. Flash 2 MB (16 ext.).	CMSIS-NN (no SIMD)	\$4–6
Arduino Nano	ATmega328P (AVR 8-bit), 1 core, 16 MHz	No	SRAM 2 KB; PSRAM no; Flash 32 KB; FPU no	None	\$10–25
STM32 (H7 ex.)	Arm Cortex-M7, 1–2 cores, 480 MHz	No	SRAM 1 MB; PSRAM ext.; Flash 2 MB; FPU DP	CMSIS-NN/DSP (no MVE)	\$8–20
STM32N6	Arm Cortex-M55 + Neural-ART NPU, 1 core, 800 MHz	No	SRAM 4.2 MB; PSRAM ext.; Flash flashless; FPU H/SP/DP	Helium SIMD + NPU 600 GOPS; ST Edge AI	\$5–12

sensor data to cloud servers, with much powerful computing capabilities, for processing and then retrieving commands from those servers in order to take physical actions. While this utilized the full capabilities of Cloud-Cluster and reduced the required specs for IoT devices, this approach suffers from critical issues:

- 1) Reduced output latency due to data + command Round Trips to and from the servers.
- 2) The need for good wifi connections which was not applicable in many cases.
- 3) Exposing security vulnerabilities and privacy concerns.
- 4) The need for developing possibly expensive server infrastructures.

These reasons were the main catalysts for EdgeML research which focussed on optimizing ML frameworks in order to run on mobile edge devices and MCUs. TinyML is a subclass of EdgeML which focusses on squeezing deep learning model on cheap, tiny MCUs through optimizing ML kernels and hardware to fit the constraints at hand.

Since then, TinyML has gained wide traction and adoption in many sectors that previously relied on CloudML. This includes Smart Home devices which benefits from object detection and face recognition DL models, smart watches and wearable devices for instantaneous detection of medical abnormalities and handling emergency situations faster, Human machine interaction specifically developing Gesture recognition software for more immersive experiences, etc.

1.2.4. TinyML Frameworks and tools

Different paradigms have been suggested and developed for TinyML development. The most popular approach for TinyML is through

1.2.5. Applications of TinyML in Bioelectric Signal Processing

2. State of The Art (SoTA)

2.0.1. EMG Signal Processing Pipelines on Microcontroller Platforms

2.0.2. EMG Signal Processing Pipelines with TinyML Frameworks

3. Materials and Methods

3.1. Dataset Description and Preprocessing

3.1.1. NinaPro DB2 Dataset

3.1.2. Preprocessing and Feature Extraction

3.2. Deep Learning Model Architecture

3.3. Hardware

3.3.1. ESP32-S3 Specifications

3.4. Quantization

Normally, Deep learning pipelines rely on floating point operations and embed their parameters (i.e weights and biases) as float32s. Unfortunately, not all MCU architectures have embedded FPU (floating point units) to handle calculations in floating point notations. In addition, chips that have dedicated FPUs in their architecture, for handling float32 data, suffer from longer processing times and resource intensive operation compared to the faster integer arithmetic units [1, 2, 3]. Furthermore, floating point numbers necessitate larger spaces in memory for tensor allocations (i.e inputs, outputs, model parameters and activations) [1].

For these reasons, different quantization schemes are widely adopted across TinyML literature. It is also the case on many MCU platforms (like the ESP32 and ESP32s3), that optimized low-level kernels for ML ops (like Convolutions and MatMuls) are exclusively written with quantization schemes, like INT8, in mind. In fact, 8-bit quantization has been shown to reduce model sizes by a factor of 4 and produce a speedup gain of 2x-3x, even 10x on specialized processors, compared to float Implementations[1].

On the other hand, quantization algorithms are not lossless since they map a continuous floating point representation into a discrete range. A degradation in final accuracy is always expected in quantized outputs. That's why quantization pipelines require careful fine-tuning to balance quality and performance.

3.4.1. Quantization Types and Schemes

Quantization schemes have been categorized into many section. In this work, we are going to outline schemes that are only relevant to TinyML research.

Data Types

By default, popular frameworks like PyTorch, Tensorflow and JAX utilize single-precision floating point numbers (float32s). Quantization schemes can then be categorized into the output data types that these float32s are converted to.

Float16 quantization uses half the space used by float32s while maintaining the floating point interface as float32s. However, float16 adoption in MCU architectures still remain rare. Most MCUs don't have dedicated kernels for handling float16s and just use float32 interfaces rendering float16 optimizations completely useless. Others are unable to handle float16 data and immediately throw exceptions.

Int8, Int16, Int32 quantization are by far the most used. Most MCU architectures are optimized for handling integer operations which makes int quantization adoption easier and more natural. Low-Level kernels usually use a mix of various integer sizes in their internal Implementation, usually inputs, outputs and weights are int8 while biases use larger formats like int32s.

Asymmetric vs symmetric quantization

symmetric quantization restricts the zero-point to 0. It only computes the scale of the floating-point values as the ration between the absolute maximum value in the floating point range and the maximum value in the quantization range. Commonly used for quantizing weights and biases as[1]. The reason why it is preferred for weights is because using asymmetric quantization (with an additional zero-point value for weights) adds an additional term to each $\mathbf{W} \cdot \mathbf{x}$ calculation which costs additional compute time during inference [3].

$$\begin{aligned}
 x_{int} &= \text{round}\left(\frac{x}{\Delta}\right) \\
 x_Q &= \text{clamp}(-N_{levels}/2, N_{levels}/2 - 1, x_{int}) && \text{if signed} \\
 x_Q &= \text{clamp}(0, N_{levels} - 1, x_{int}) && \text{if un-signed} \\
 x_{out} &= x_Q \Delta && \text{(de-quantization)}
 \end{aligned} \tag{3.1}$$

Asymmetric quantization calculates an optimal new zero-point value to map the floating point zero to it. Preferred for activations.

Quantization Aware Training (QAT) vs Post Training Quantization (PTQ)

$$\begin{aligned}
 x_{int} &= \text{round}\left(\frac{x}{\Delta}\right) + z \\
 x_Q &= \text{clamp}(0, N_{levels} - 1, x_{int}) \\
 x_{float} &= (x_Q - z) \Delta \quad (\text{de-quantization})
 \end{aligned} \tag{3.2}$$

Post Training Quantization (PTQ) works on quantization pre-trained DL models, trained using default floating-point numbers. The quantizing algorithm takes pre-trained weights and biases and calculates the appropriate scaling and shifting parameters. For activations, it uses a representative dataset (representing a sample of input values in floating space) and propagates it through the model to get a closer statistical model of how values fluctuate in the actual model in order to map it to the quantized range. PTQs are widely adopted in pipelines that aim at deploying normal, pre-trained models, usually not co-designed for TinyML use, on Edge device without having to re-train the model [1, 3].

Quantization Aware Training (QAT) involves having quantization in mind during the training of the actual model, with simulated quantization applied at every step of gradient descent so that the model learns to compensate for quantization noise. The weights are maintained in floating point and updated using floating-point gradients, while a simulated quantization operation (fake quantization) emulates the effect of integer inference during the forward and backward passes. While this method can provide higher accuracies than PTQ [1, 3], it necessitates active co-design of the DL model and is not viable for quantizing all available models, since it requires retraining in the presence of simulated quantization, which demands additional training resources and time-consuming retraining.

$$\begin{aligned}
 w_{float} &= w_{float} - \eta \frac{\partial L}{\partial w_{out}} \cdot I_{w_{out} \in (w_{min}, w_{max})} \\
 w_{out} &= \text{SimQuant}(w_{float})
 \end{aligned} \tag{3.3}$$

Hybrid vs Full quantization

Hybrid quantization is defined as quantizing the inputs, weights and biases only while leaving intermediate activations as floating point. While this can improve final accuracy since a full quantization is not adopted and activations remain in their full floating point resolution, many MCU platforms and frameworks do not fully support mixed precision computation. In addition, Hybrid models produced the overhead of having to requantize activations after each neural layer which can detrimentally impact the final inference time and the needed allocation space for larger float32 intermediate values.

Full quantization involves the quantization of the entire Pipeline eliminating the need for intermittent requantization steps and produces more faster and more lightweight models since there are not intermediate floats. However, for full quantization, more overhead is needed for calculating additional quantization parameters for activation layers and , depending on the quantization scheme used, may require a set of representative data to calculate those quantization parameters.s

Quantization granularity

Quantization can be done with varying degrees of resolution at the cost of producing an increasing amount of quantization parameters and settings.

Per Tensor quantization involves calculating scaling and shifting params for each tensor involved in the pipeline. This scheme is preferred for activations [1].

Per Channel quantization involves having a set of quantization params per each channel in a tensor. This presents a more granular quantization scheme as it accounts for distribution differences between individual channels in the data at hand. This is the recommended method for stored weights [1, 3].

3.4.2. Quantization Scheme Used

In this work, we opted for a PTQ quantization scheme. In order to comply with ESP-NN backend functions, we use full int8 quantization across the entire model with int32 accumulators and int32 for biases as presented here [2]. Since ESP-NN uses the same quantization schemes as TFlite runtime, we are using per tensor asymmetric quantization for inputs and activations while using symmetric per channels quantization for both weights and biases. Similar 8-bit PTQ schemes, with per channel quantization for weights and per layer quantization of activations, have shown accuracies within 2 percent of floating point networks, particularly for CNN architectures [1].

3.5. Software and Tools

3.5.1. ESP-NN Library

The ESP-NN library contains optimized NN (Neural Network) functions for various Espressif chips. They are the backend used by TensorFlow Lite Micro on esp32 chips. It supports ESP32, ESP32-S3, ESP32-P4 and ESP32-C3 chips. While it provides generic optimizations for most chips, it provide assembly optimized versions for ML dedicated chips like ESP32-S3.

The optimized ESP-NN libraries on ESP32-S3 chip are only dedicated for Int8 quantized operations. For matrix multiplications, the core arithmetic operations behind fully connected and LSTM layers, 2 functions are available for use. `esp_nn_fully_connected` which is used for per tensor weight quantization and the `esp_nn_fully_connected_per_ch` variant for per channel quantized weights.

Similar to [2], ESP-NN uses int32 accumulators for accumulating the intermediate products in matmul operations. This follows directly from the integer-arithmetic-only formulation of [2], where each real value r is represented through an affine mapping $r = S(q - Z)$ from its quantized integer q , with scale S and zero-point Z . Under this mapping, a quantized matrix product reduces to:

$$q_3^{(i,k)} = Z_3 + M \sum_{j=1}^N (q_1^{(i,j)} - Z_1)(q_2^{(j,k)} - Z_2), \quad (3.4)$$

where the summation is performed entirely in integers and held in a wide (int32) accumulator, exactly as ESP-NN does. The only non-integer term is the multiplier $M = S_1 S_2 / S_3$, which depends only on the fixed tensor scales and is therefore constant at inference. Since $M \in (0, 1)$, it is expressed in the fixed-point form $M = 2^{-n} M_0$ with $M_0 \in [0.5, 1)$. This is precisely how the rescaling is discretized into the kernel’s two integer parameters: M_0 becomes the fixed-point multiplier `out_mult`, and the factor 2^{-n} becomes a rounding right-shift by `out_shift`. Requantization thus reduces to one integer multiply followed by a shift, keeping the matmul pipeline entirely integer-only.

3.5.2. LiteRT (Tensorflow Lite) Framework

3.5.3. LabStreaming Layer (LSL) for Data Acquisition and Real-Time Processing

4. Results

4.1. Comparison of ML Ops Performance on the ESP32-S3 using Random Weights under different Quantization Schemes

4.2. Comparison of different Model Configurations

Table 4.1.: Dense layers performance on the ESP32-S3.

	PSRAM				No PSRAM			
	Float32		Int8		Float32		Int8	
	Inf (μ s)	MSE	Inf (μ s)	MSE	Inf (μ s)	MSE	Inf (μ s)	MSE
Dense(25→2, L=2)	142	0	189	0	134	0	192	0
Dense(25→2, L=3)	166	0	225	0	161	0	225	0
Dense(25→2, L=4)	174	0	237	0.0003	169	0	237	0.0003
Dense(25→2, L=5)	184	0	249	0.0012	178	0	248	0.0012
Dense(25→2, L=6)	192	0	267	0.0040	195	0	258	0.0040
Dense(25→2, L=7)	199	0	273	0	194	0	271	0
Dense(25→2, L=8)	209	0	283	0.0034	201	0	280	0.0034
Dense(25→2, L=9)	240	0	302	0.0216	212	0	293	0.0216
Dense(25→2, L=10)	228	0	320	0.0019	221	0	305	0.0019
Dense(25→2, L=11)	236	0	321	0.0202	228	0	315	0.0202
Dense(50→2, L=2)	141	0	206	0	137	0	206	0
Dense(100→2, L=2)	150	0	237	0	145	0	237	0
Dense(150→2, L=2)	163	0	269	0	151	0	268	0
Dense(200→2, L=2)	177	0	297	0	159	0	299	0
Dense(250→2, L=2)	178	0	334	0.0002	165	0	330	0.0002
Dense(300→2, L=2)	185	0	361	0	173	0	361	0
Dense(350→2, L=2)	208	0	391	0.0009	180	0	392	0.0009
Dense(400→2, L=2)	214	0	426	0	187	0	422	0
Dense(450→2, L=2)	216	0	456	0	194	0	453	0
Dense(500→2, L=2)	253	0	493	0.0002	206	0	490	0.0002
Dense(25→1, L=2)	140	0	194	0	135	0	196	0
Dense(25→5, L=2)	140	0	196	0.0002	134	0	198	0.0002
Dense(25→10, L=2)	142	0	203	0.0001	137	0	205	0.0001
Dense(25→20, L=2)	162	0	217	0.0001	145	0	219	0.0001
Dense(25→25, L=2)	154	0	224	0.0001	153	0	226	0.0001
Dense(25→30, L=2)	155	0	234	0.0002	152	0	233	0.0002
Dense(25→35, L=2)	177	0	239	0.0001	156	0	240	0.0001
Dense(25→40, L=2)	180	0	245	0.0002	160	0	247	0.0002
Dense(25→45, L=2)	178	0	256	0.0002	163	0	254	0.0002
Dense(25→50, L=2)	180	0	260	0.0001	167	0	261	0.0001

Table 4.2.: Dense layers performance on the ESP32-S3.

	PSRAM				No PSRAM			
	Float32		Int8		Float32		Int8	
	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)
Dense(25→2, L=2)	1.1	0.5	1.1	0.5	1.1	0.5	1.1	0.5
Dense(25→2, L=3)	2.3	0.7	1.7	0.7	2.3	0.7	1.7	0.7
Dense(25→2, L=4)	3.2	0.8	2.4	0.9	3.2	0.8	2.4	0.9
Dense(25→2, L=5)	4.0	1.0	3.0	1.0	4.0	1.0	3.0	1.1
Dense(25→2, L=6)	4.8	1.1	3.6	1.3	4.8	1.1	3.6	1.3
Dense(25→2, L=7)	5.6	1.2	4.3	1.4	5.6	1.2	4.3	1.4
Dense(25→2, L=8)	6.4	1.3	4.9	1.6	6.4	1.3	4.9	1.6
Dense(25→2, L=9)	7.2	1.5	5.6	1.9	7.2	1.5	5.6	1.9
Dense(25→2, L=10)	8.1	1.6	6.2	2.1	8.1	1.6	6.2	2.1
Dense(25→2, L=11)	8.8	1.7	6.8	2.2	8.8	1.7	6.8	2.2
Dense(50→2, L=2)	1.3	0.6	1.1	0.5	1.3	0.6	1.1	0.5
Dense(100→2, L=2)	1.7	0.8	1.2	0.6	1.7	0.8	1.2	0.6
Dense(150→2, L=2)	2.1	1.0	1.3	0.6	2.1	1.0	1.3	0.6
Dense(200→2, L=2)	2.5	1.2	1.4	0.7	2.5	1.2	1.4	0.6
Dense(250→2, L=2)	2.9	1.4	1.5	0.7	2.9	1.4	1.5	0.7
Dense(300→2, L=2)	3.2	1.6	1.6	0.7	3.2	1.5	1.6	0.7
Dense(350→2, L=2)	3.6	1.8	1.7	0.8	3.6	1.8	1.7	0.8
Dense(400→2, L=2)	4.0	1.9	1.8	0.8	4.0	1.9	1.8	0.8
Dense(450→2, L=2)	4.4	2.1	1.9	0.9	4.4	2.1	1.9	0.9
Dense(500→2, L=2)	4.8	2.3	2.0	0.9	4.8	2.3	2.0	0.9
Dense(25→1, L=2)	1.0	0.5	1.1	0.5	1.0	0.5	1.1	0.4
Dense(25→5, L=2)	1.4	0.5	1.2	0.5	1.4	0.5	1.2	0.5
Dense(25→10, L=2)	1.9	0.5	1.4	0.5	1.9	0.5	1.4	0.5
Dense(25→20, L=2)	2.9	0.6	1.8	0.6	2.9	0.5	1.8	0.6
Dense(25→25, L=2)	3.3	0.6	1.9	0.7	3.3	0.6	1.9	0.7
Dense(25→30, L=2)	3.8	0.6	2.1	0.7	3.8	0.6	2.1	0.7
Dense(25→35, L=2)	4.3	0.6	2.3	0.8	4.3	0.6	2.3	0.8
Dense(25→40, L=2)	4.8	0.6	2.5	0.8	4.8	0.6	2.5	0.8
Dense(25→45, L=2)	5.3	0.7	2.6	0.9	5.3	0.7	2.6	0.8
Dense(25→50, L=2)	5.8	0.7	2.8	0.9	5.8	0.7	2.8	0.9

Table 4.3.: Conv2D layers performance on the ESP32-S3.

	PSRAM				No PSRAM			
	Float32		Int8		Float32		Int8	
	Inf (μ s)	MSE	Inf (μ s)	MSE	Inf (μ s)	MSE	Inf (μ s)	MSE
Conv(in=1, out=10, L=1, k=(1, 5))	1813	0	1198	0.0003	1574	0	1191	0.0003
Conv(in=1, out=10, L=2, k=(1, 5))	2933	0	1768	0.0005	2908	0	1755	0.0005
Conv(in=1, out=10, L=3, k=(1, 5))	2807	0	1701	0.0036	2785	0	1680	0.0036
Conv(in=1, out=10, L=4, k=(1, 5))	2464	0	1556	0.0022	2453	0	1548	0.0022
Conv(in=1, out=10, L=1, k=(2, 2))	1636	0	1133	0.0002	1472	0	1125	0.0002
Conv(in=1, out=10, L=1, k=(1, 10))	2338	0	1308	0.0002	2260	0	1305	0.0002
Conv(in=1, out=10, L=1, k=(1, 20))	3386	0	1458	0.0002	3297	0	1436	0.0002
Conv(in=1, out=10, L=1, k=(1, 30))	4118	0	1526	0.0002	4014	0	1514	0.0002
Conv(in=1, out=1, L=1, k=(1, 5))	389	0	448	0.0002	386	0	445	0.0002
Conv(in=1, out=20, L=1, k=(1, 5))	3439	0	1433	0.0003	2943	0	1353	0.0003
Conv(in=1, out=30, L=1, k=(1, 5))	5077	0	2694	0.0003	4253	0	2469	0.0003

Table 4.4.: Conv2D layers performance on the ESP32-S3.

	PSRAM				No PSRAM			
	Float32		Int8		Float32		Int8	
	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)
Conv(in=1, out=10, L=1, k=(1, 5))	2.3	7.9	2.6	2.6	2.3	7.9	2.6	2.6
Conv(in=1, out=10, L=2, k=(1, 5))	3.6	4.0	3.8	3.2	3.6	4.0	3.8	3.2
Conv(in=1, out=10, L=3, k=(1, 5))	4.9	2.9	4.8	3.4	4.9	2.9	4.8	3.4
Conv(in=1, out=10, L=4, k=(1, 5))	6.0	3.0	5.8	3.5	6.0	3.0	5.8	3.5
Conv(in=1, out=10, L=1, k=(2, 2))	2.3	7.1	2.6	2.4	2.3	7.1	2.6	2.4
Conv(in=1, out=10, L=1, k=(1, 10))	2.5	7.9	2.6	2.6	2.5	7.9	2.6	2.6
Conv(in=1, out=10, L=1, k=(1, 20))	2.9	7.1	2.7	2.4	2.9	7.1	2.7	2.4
Conv(in=1, out=10, L=1, k=(1, 30))	3.3	6.3	2.8	2.2	3.3	6.3	2.8	2.2
Conv(in=1, out=1, L=1, k=(1, 5))	1.7	1.4	1.9	0.9	1.7	1.4	1.9	0.9
Conv(in=1, out=20, L=1, k=(1, 5))	2.5	15.8	2.9	4.6	2.5	15.8	2.9	4.6
Conv(in=1, out=30, L=1, k=(1, 5))	2.8	23.4	3.2	6.6	2.8	23.4	3.2	6.5

5. Discussion

5.1. Software Challenges

5.2. LSTM Compatibility and Overhead

6. Conclusion

6.1. Conclusion

6.2. Limitations

6.3. Future Work

A. Additionally

List of Figures

List of Tables

- 1.1. Comparison of Commercially Available MCUs for Edge ML Applications . . 3
- 4.1. Dense layers performance on the ESP32-S3. 14
- 4.2. Dense layers performance on the ESP32-S3. 15
- 4.3. Conv2D layers performance on the ESP32-S3. 15
- 4.4. Conv2D layers performance on the ESP32-S3. 16

Bibliography

- [1] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” Jun. 2018.
- [2] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam and D. Kalenichenko, “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” Dec. 2017.
- [3] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen and T. Blankevoort, “A White Paper on Neural Network Quantization,” Jun. 2021.

Declaration

Herewith, I declare that I have developed and written the enclosed thesis entirely by myself and that I have not used sources or means except those declared.

This thesis has not been submitted to any other authority to achieve an academic grading and has not been published elsewhere.

Stuttgart, TBD Date of sign.

 Andrew Fady Fahmy